

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе №8
«Графы»

Выполнила работу:
Абаянцева Евгения
Академическая группа №J3110
Принято:
...

Санкт-Петербург
2024 г.

Введение

Цель – написать код для поиска наикратчайшего пути.

Задача: реализовать четыре алгоритма (bfs, dfs, алгоритм Дэйкстры и A^*) для поиска наикратчайшего пути и исследовать их.

Теоретическая подготовка

Данная задача предполагает решение с использованием графов. Используемые типы данных для этой задачи – целые числа (тип `int`), кортежи (тип `vector`), строки (тип `string`), словарь (`unordered_map`), вещественные числа (тип `double`), символы (тип `char`).

Реализация

Для этой задачи требуются следующие библиотеки:

1. Библиотека `iostream` – эта основная библиотека C++ для ввода и вывода текста с консоли;
2. Библиотека `unordered_map` – эта библиотека определяет шаблон класса `unordered_map`;
3. Библиотека `vector` – эта библиотека определяет вектор шаблона класса контейнера и несколько вспомогательных шаблонов;
4. Библиотека `stack` – эта библиотека определяет стек шаблонов класса и два вспомогательных шаблона;
5. Библиотека `queue` – эта библиотека определяет шаблоны классов `priority_queue` и очереди и несколько вспомогательных шаблонов;
6. Библиотека `cmath` – эта библиотека включает заголовок `<math.h>` стандартной библиотеки C и добавляет связанные имена в `std` пространство имен;
7. Библиотека `limits` – эта библиотека определяет шаблон `numeric_limits` класса и два перечисления, касающиеся представлений с плавающей запятой и округления;
8. Библиотека `fstream` – эта библиотека определяет несколько классов, поддерживающих операции `iostreams` для последовательностей, хранящихся во внешних файлах;

9. Библиотека `sstream` – эта функция определяет несколько шаблонов классов, поддерживающих операции `istream` с последовательности, хранящиеся в выделенном объекте массива.
10. Библиотека `chrono` – эта библиотека отвечает за работу со временем и датой;
11. Библиотека `functional` – эта библиотека определяет функции стандартной библиотеки C++, которые помогают создавать объекты функций, также известные как воронки и их привязыватели;
12. Библиотека `algorithm` – эта библиотека определяет функции шаблона контейнера стандартной библиотеки C++, которые выполняют алгоритмы.

Реализация была сделана путём создания нескольких функций в классе `Graph`, представленных ниже:

1. Функции `parse_data`;
2. Функции `search`;
3. Функций `bfs`, `dfs`, алгоритма Дэйкстры, A^* .

Рассмотрим каждую из них.

Представленная функция `parse_data` была реализована для разборки входной строки для построения графа:

```
1 void parse_data(const string& data) {
2     stringstream ss(data);
3     string line;
4
5     while (getline(ss, line)) {
6         stringstream lineStream(line);
7         double lon1, lat1, lon2, lat2, weight;
8         char s;
9
10        if (lineStream >> lon1 >> s >> lat1 && s == ',' && lineStream.get() == '
11        :') {
12            Node node1{ lon1, lat1 };
13            while (lineStream >> lon2 >> s >> lat2 >> s >> weight) {
14                Node node2{ lon2, lat2 };
15                adjList[node1].emplace_back(node2, weight);
16                adjList[node2].emplace_back(node1, weight);
17                if (!(lineStream >> s) || s != ';')
18                    break;
19            }
20        }
21    }
```

Листинг 1 – функция `parse_data`

Представленная функция search была придумана и использована для упрощения программы (по структуре кода) для алгоритмов bfs, алгоритма Дэйкстры и A*. Она представлена ниже:

```

1 pair<double, vector<Node>> search(const Node& start, const Node& end, function<
    bool(priority_queue<pair<double, Node>, vector<pair<double, Node>>, greater
    <>>&, const Node&, double, double)> heuristic = nullptr) {
2     using pq_type = priority_queue<pair<double, Node>, vector<pair<double, Node
    >>, greater<>>;
3     pq_type pq;
4     unordered_map<Node, double> distances;
5     unordered_map<Node, Node> parent;
6     unordered_map<Node, bool> visited;
7
8     for (const auto& pair : adjList) distances[pair.first] = numeric_limits<
    double>::infinity();
9     distances[start] = 0.0;
10    pq.push({ 0.0, start });
11    parent[start] = start;
12
13    while (!pq.empty()) {
14        double currentDistance = pq.top().first;
15        Node current = pq.top().second;
16        pq.pop();
17
18        if (current == end) {
19            vector<Node> path;
20            for (Node at = end; at != start; at = parent[at]) path.push_back
    (at);
21            path.push_back(start);
22            reverse(path.begin(), path.end());
23            return { currentDistance, path };
24        }
25
26        if (visited[current]) continue;
27        visited[current] = true;
28
29        for (const auto& neighbor_pair : adjList[current]) {
30            const Node& neighbor = neighbor_pair.first;
31            double weight = neighbor_pair.second;
32            double newDist = currentDistance + weight;
33
34            if (newDist < distances[neighbor]) {
35                distances[neighbor] = newDist;
36                parent[neighbor] = current;
37
38                double priority = newDist;
39                if (heuristic) priority += heuristic(pq, neighbor, newDist,
    currentDistance);
40
41                pq.push({ priority, neighbor });
42            }
43        }
44    }
45    return { numeric_limits<double>::infinity(), {} };
46 }

```

Листинг 2 – функция search

Поэтому следующие функции представляются в виде:

Функция bfs:

```
1 pair<double, vector<Node>> bfs(const Node& start, const Node& end) {  
2     return search(start, end);  
3 }
```

Листинг 3 – функция bfs

Функция алгоритма Дэйкстры:

```
1 pair<double, vector<Node>> dijkstra(const Node& start, const Node& end) {  
2     return search(start, end);  
3 }
```

Листинг 4 – функция алгоритма Дэйкстры

Функция A*:

```
1 pair<double, vector<Node>> a_star(const Node& start, const Node& end) {  
2     auto heuristic = [&end](auto&, const Node& node, double, double) {  
3         return sqrt(pow(node.leng - end.leng, 2) + pow(node.widt - end.widt, 2))  
4     };  
5     return search(start, end, heuristic);  
6 }
```

Листинг 5 – функция A*

И, следовательно, последняя функция dfs, для которой использовался СТЭК:

```
1 pair<double, vector<Node>> dfs(const Node& start, const Node& end) {  
2     stack<Node> s;  
3     unordered_map<Node, bool> visited;  
4     unordered_map<Node, Node> parent;  
5     unordered_map<Node, double> distances;  
6  
7     for (const auto& pair : adjList) distances[pair.first] = numeric_limits<  
8         double>::infinity();  
9     distances[start] = 0.0;  
10  
11     s.push(start);  
12     parent[start] = start;  
13     while (!s.empty()) {  
14         Node current = s.top();  
15         s.pop();  
16  
17         if (current == end) {  
18             vector<Node> path;  
19             for (Node at = end; at != start; at = parent[at]) path.push_back(at)  
20             ;  
21             path.push_back(start);  
22             reverse(path.begin(), path.end());  
23             return { distances[end], path };  
24  
25             if (visited[current]) continue;  
26             visited[current] = true;  
27  
28             for (const auto& neighbor_pair : adjList[current]) {  
29                 const Node& neighbor = neighbor_pair.first;
```

```

30         double weight = neighbor_pair.second;
31
32         if (!visited[neighbor]) {
33             distances[neighbor] = distances[current] + weight;
34             parent[neighbor] = current;
35             s.push(neighbor);
36         }
37     }
38 }
39
40 // Если путь не найден
41 return { numeric_limits<double>::infinity(), {} };
42 }

```

Листинг 6 – функция dfs

В конце выдаётся массив, который содержит максимальные цифры массива, не нарушая исходного порядка в двух переданных массивах.

Особенность реализации: для функции dfs используется стек, для функций bfs, алгоритма Дэйкстры и A^* используется отдельная функция. Также для функции A^* используется эвристическая функция.

Экспериментальная часть

1. Память bfs: 116 байт на n узлов графа.
Память dfs: 116 байт на n узлов графа.
Память алгоритма Дэйкстры: 116 байт на n узлов графа.
Память A^* : 116 байт на n узлов графа.
2. Сложность bfs: $O(V + E)$.
Сложность dfs: $O(V + E)$.
Сложность алгоритма Дэйкстры: $O(V + E)$.
Сложность A^* : $O(V + E)$, где V – количество узлов, E – количество рёбер.

Заключение

Для исследования или нахождения пути в разреженных графах, где кратчайший путь не важен, лучше использовать dfs.

Для невзвешенных графов, где важен кратчайший путь, лучше использовать bfs.

Для взвешенных графов с положительными весами на рёбрах лучше использовать алгоритм Дэйкстры.

Для больших графов, где важен быстрый поиск пути с помощью эвристики, лучше использовать A^* .

В конце приложения представлена таблица со сравнениями всех

алгоритмов.

Приложения

```
1 #include <iostream>
2 #include <unordered_map>
3 #include <vector>
4 #include <stack>
5 #include <queue>
6 #include <cmath>
7 #include <limits>
8 #include <fstream>
9 #include <sstream>
10 #include <chrono>
11 #include <functional>
12 #include <algorithm>
13
14 using namespace std;
15
16 struct Node {
17     double leng, widt;
18
19     bool operator==(const Node& other) const {
20         return leng == other.leng && widt == other.widt;
21     }
22
23     bool operator!=(const Node& other) const {
24         return !(*this == other);
25     }
26
27     bool operator<(const Node& other) const {
28         return tie(leng, widt) < tie(other.leng, other.widt);
29     }
30 };
31
32 namespace std {
33     template <>
34     struct hash<Node> {
35         size_t operator()(const Node& node) const {
36             return hash<double>()(node.leng) ^ (hash<double>()(node.widt) << 1);
37         }
38     };
39 }
40
41 class Graph {
42 public:
43     void parse_data(const string& data) {
44         stringstream ss(data);
45         string line;
46
47         while (getline(ss, line)) {
48             stringstream lineStream(line);
49             double lon1, lat1, lon2, lat2, weight;
50             char s;
51
52             if (lineStream >> lon1 >> s >> lat1 && s == ',' && lineStream.get() == ':')
53                 Node node1{ lon1, lat1 };
```

```

54     while (lineStream >> lon2 >> s >> lat2 >> s >> weight) {
55         Node node2{ lon2, lat2 };
56         adjList[node1].emplace_back(node2, weight);
57         adjList[node2].emplace_back(node1, weight);
58         if (!(lineStream >> s) || s != ';') break;
59     }
60 }
61 }
62 }
63
64 pair<double, vector<Node>> search(const Node& start, const Node& end,
65     function<bool(priority_queue<pair<double, Node>, vector<pair<double, Node>>,
66         greater<>>&, const Node&, double, double)> heuristic = nullptr) {
67
68     using pq_type = priority_queue<pair<double, Node>, vector<pair<double, Node
69 >>, greater<>>;
70     pq_type pq;
71     unordered_map<Node, double> distances;
72     unordered_map<Node, Node> parent;
73     unordered_map<Node, bool> visited;
74
75     for (const auto& pair : adjList) distances[pair.first] = numeric_limits<
76 double>::infinity();
77     distances[start] = 0.0;
78     pq.push({ 0.0, start });
79     parent[start] = start;
80
81     while (!pq.empty()) {
82         double currentDistance = pq.top().first;
83         Node current = pq.top().second;
84         pq.pop();
85
86         if (current == end) {
87             vector<Node> path;
88             for (Node at = end; at != start; at = parent[at]) path.push_back(at);
89             path.push_back(start);
90             reverse(path.begin(), path.end());
91             return { currentDistance, path };
92         }
93
94         if (visited[current]) continue;
95         visited[current] = true;
96
97         for (const auto& neighbor_pair : adjList[current]) {
98             const Node& neighbor = neighbor_pair.first;
99             double weight = neighbor_pair.second;
100             double newDist = currentDistance + weight;
101
102             if (newDist < distances[neighbor]) {
103                 distances[neighbor] = newDist;
104                 parent[neighbor] = current;
105
106                 double priority = newDist;
107                 if (heuristic) priority += heuristic(pq, neighbor, newDist,
108 currentDistance);
109
110                 pq.push({ priority, neighbor });
111             }
112         }
113     }
114 }

```

```

110     return { numeric_limits<double>::infinity(), {} };
111 }
112
113 pair<double, vector<Node>> bfs(const Node& start, const Node& end) {
114     return search(start, end);
115 }
116
117 pair<double, vector<Node>> dfs(const Node& start, const Node& end) {
118     stack<Node> s;
119     unordered_map<Node, bool> visited;
120     unordered_map<Node, Node> parent;
121     unordered_map<Node, double> distances;
122
123     for (const auto& pair : adjList) distances[pair.first] = numeric_limits<
double>::infinity();
124     distances[start] = 0.0;
125
126     s.push(start);
127     parent[start] = start;
128
129     while (!s.empty()) {
130         Node current = s.top();
131         s.pop();
132
133         if (current == end) {
134             vector<Node> path;
135             for (Node at = end; at != start; at = parent[at]) path.push_back(at);
136             path.push_back(start);
137             reverse(path.begin(), path.end());
138             return { distances[end], path };
139         }
140
141         if (visited[current]) continue;
142         visited[current] = true;
143
144         for (const auto& neighbor_pair : adjList[current]) {
145             const Node& neighbor = neighbor_pair.first;
146             double weight = neighbor_pair.second;
147
148             if (!visited[neighbor]) {
149                 distances[neighbor] = distances[current] + weight;
150                 parent[neighbor] = current;
151                 s.push(neighbor);
152             }
153         }
154     }
155
156     // Если путь не найден
157     return { numeric_limits<double>::infinity(), {} };
158 }
159
160
161 pair<double, vector<Node>> dijkstra(const Node& start, const Node& end) {
162     return search(start, end);
163 }
164
165 pair<double, vector<Node>> a_star(const Node& start, const Node& end) {
166     auto heuristic = [&end](auto&, const Node& node, double, double) {
167         return sqrt(pow(node.leng - end.leng, 2) + pow(node.widt - end.widt, 2));
168     };

```

```

169     return search(start, end, heuristic);
170 }
171
172 private:
173     unordered_map<Node, vector<pair<Node, double>>> adjList;
174 };
175
176 void test_algorithms(Graph& graph, const Node& start, const Node& end, double
    expectedDistance) {
177     auto bfs_result = graph.bfs(start, end);
178     cout << "BFS: Distance: " << bfs_result.first
179         << " (Expected: " << expectedDistance << ")\n";
180
181     auto dfs_result = graph.dfs(start, end);
182     cout << "DFS: Distance: " << dfs_result.first
183         << " (Expected: " << expectedDistance << ")\n";
184
185     auto dijkstra_result = graph.dijkstra(start, end);
186     cout << "Dijkstra: Distance: " << dijkstra_result.first
187         << " (Expected: " << expectedDistance << ")\n";
188
189     auto a_star_result = graph.a_star(start, end);
190     cout << "A*: Distance: " << a_star_result.first
191         << " (Expected: " << expectedDistance << ")\n";
192 }
193
194 int main_test() {
195     Graph graph;
196
197     ifstream inputFile("spb_graph.txt");
198     if (!inputFile.is_open()) {
199         cerr << "Не удалось открыть файл!" << endl;
200         return 1;
201     }
202     cout << "File open!" << endl;
203
204     stringstream buffer;
205     buffer << inputFile.rdbuf();
206     string data = buffer.str();
207     graph.parse_data(data);
208
209     Node start{ 30.216413, 59.9554621 };
210     Node end{ 30.4680341, 59.8621847 };
211
212     auto start_bfs = chrono::steady_clock::now();
213     auto bfs_result = graph.bfs(start, end);
214     auto end_bfs = chrono::steady_clock::now();
215     chrono::duration<double> elapsed_bfs = end_bfs - start_bfs;
216     cout << "Algorithm BFS:\n Distance: " << bfs_result.first << " Edges: " <<
        bfs_result.first << " Time: "
217         << elapsed_bfs.count() << " ms" << endl;
218
219     auto start_dfs = chrono::steady_clock::now();
220     auto dfs_result = graph.dfs(start, end);
221     auto end_dfs = chrono::steady_clock::now();
222     chrono::duration<double> elapsed_dfs = end_dfs - start_dfs;
223     cout << "Algorithm DFS:\n Distance: " << dfs_result.first << " Edges: " <<
        dfs_result.first << " Time: "
224         << elapsed_dfs.count() << " ms" << endl;
225

```

```

226 auto start_dijkstra = chrono::steady_clock::now();
227 auto dijkstra_result = graph.dijkstra(start, end);
228 auto end_dijkstra = chrono::steady_clock::now();
229 chrono::duration<double> elapsed_dijkstra = end_dijkstra - start_dijkstra;
230 cout << "Algorithm Dijkstra:\n Distance: " << dijkstra_result.first << " Time:
    "
231     << elapsed_dijkstra.count() << " ms" << endl;
232
233 auto start_a_star = chrono::steady_clock::now();
234 auto a_star_result = graph.a_star(start, end);
235 auto end_a_star = chrono::steady_clock::now();
236 chrono::duration<double> elapsed_a_star = end_a_star - start_a_star;
237 cout << "Algorithm A*:\n Distance: " << a_star_result.first << " Time: "
238     << elapsed_a_star.count() << " ms" << endl;
239
240 return 0;
241 }
242
243 int main() {
244     Graph graph;
245
246     string data = R"(
247         0.0,0.0:1.0,1.0,1.41;2.0,0.0,2.0
248         1.0,1.0:2.0,0.0,1.0;3.0,1.0,1.41
249         3.0,1.0:0.0,0.0,3.0
250     )";
251     graph.parse_data(data);
252
253     Node start{ 0.0, 0.0 };
254     Node end{ 3.0, 1.0 };
255     test_algorithms(graph, start, end, 3.0);
256
257     data.clear();
258
259     main_test();
260
261     return 0;
262 }

```

Листинг 7 – код файла lab-8.cpp

Время и дистанция для каждого алгоритма:

```

Algorithm BFS:
Distance: 0.286903 Edges: 0.286903 Time: 28.9035 ms
Algorithm DFS:
Distance: 21.5228 Edges: 21.5228 Time: 26.1609 ms
Algorithm Dijkstra:
Distance: 0.286903 Time: 26.5669 ms
Algorithm A*:
Distance: 0.286903 Time: 16.2282 ms

```

Рис. 1 – время и дистанция

Таблица со сравнениями алгоритмов:

Таблица 1 – Сравнение алгоритмов

Алгоритм	DFS	BFS	Алгоритм Дэйкстры	A*
Кратчайший путь	-	+	+	+
Эвристика	-	-	-	+
Подходит для	Обход всех путей	Невзве- шенные графы	Взвешенные графы	Большие графы, реальное время
Недостатки	Может зацик- литься	Требуется больше памяти	Медленнее A*	Зависимость от эвристики